

Resume Materi Perkuliahan
Perancangan dan Analisis Algoritma (A)
Pertemuan ke-7

Disusun Oleh:
Gilbran Mahdavia Raja
NRP: 5025241134

Institut Teknologi Sepuluh Nopember
Surabaya
2026

Daftar Isi

1	Pendahuluan: Kapan Dynamic Programming Digunakan	2
2	Studi Kasus 1: E-Olymp 5081 – Lanterns	2
2.1	Deskripsi Masalah	2
2.2	Definisi State DP	3
2.3	Penurunan Relasi Rekurensi	3
2.4	Verifikasi Manual ($n = 3, k = 3$)	3
2.5	Kode Solusi (C)	4
2.6	Analisis Kompleksitas	4
3	Studi Kasus 2: SPOJ AU7_5 – Students	4
3.1	Deskripsi Masalah	4
3.2	Pendekatan 1: DP 2-State	5
3.2.1	Definisi State	5
3.2.2	Logika Transisi	5
3.2.3	Tabulasi DP 2-State ($N = 4, K = 2$)	5
3.2.4	Kode DP 2-State (C++)	5
3.3	Pendekatan 2: DP 1-State (Optimasi)	6
3.3.1	Motivasi Penyederhanaan	6
3.3.2	Bukti Nilai Awal ($dp[i] = i + 1$ saat $i - k - 1 \leq 0$)	7
3.3.3	Perbandingan Jumlah Entri	7
3.3.4	Tabulasi DP 1-State ($N = 4, K = 2$)	7
3.3.5	Kode DP 1-State (C)	7
3.4	Perbandingan Dua Pendekatan	8
3.5	Kaitan Antar Studi Kasus	8
4	Sintesis: Pola Berpikir Pertemuan 7	8
4.1	Hierarki Teknik yang Dipelajari	8
5	Checklist Konsep yang Harus dikuasai	9
6	Kesimpulan	9
7	References	10
8	Bukti Submission	11

1 Pendahuluan: Kapan Dynamic Programming Digunakan

Tidak semua permasalahan bisa diselesaikan dengan *Divide and Conquer* karena tidak selalu ada *closed form solution*. **Dynamic Programming (DP)** hadir sebagai solusi untuk masalah yang memiliki dua sifat kunci berikut.

Definisi 1.1 (Overlapping Subproblems). *Sebuah masalah memiliki **overlapping subproblems** apabila submasalah yang lebih kecil dihitung berulang kali selama proses rekursi. DP mengeliminasi pemborosan ini dengan menyimpan hasil ke dalam tabel (memoization atau tabulation).*

Definisi 1.2 (Optimal Substructure). *Sebuah masalah memiliki **optimal substructure** apabila solusi optimal keseluruhan dapat dibangun dari solusi optimal submasalah-submasalahnya.*

Ada dua teknik penyimpanan yang umum dipakai dalam DP:

Tabel 1: Perbandingan teknik memoization dan tabulation

Teknik	Arah Komputasi	Keterangan
Memoization (Top-Down)	Rekursif	Hitung saat dibutuhkan, simpan hasilnya
Tabulation (Bottom-Up)	Iteratif	Isi tabel dari kasus terkecil ke besar

Alur Berpikir Umum Soal DP

Pahami Soal → Definisikan State → Turunkan Rekurensi → Tentukan Basis → Verifikasi Manual → Kode Efisien

Pertemuan ke-7 ini membahas dua studi kasus utama:

1. **E-Olymp 5081 – Lanterns:** Menghitung konfigurasi nyala/mati lampu yang aman, dengan rekurensi DP satu dimensi yang melibatkan pengurangan modular.
2. **SPOJ AU7_5 – Students:** Menghitung barisan siswa valid, diselesaikan dengan dua pendekatan DP (2-state dan 1-state yang lebih efisien).

Pelajaran Utama Pertemuan 7

DP bukan tentang menghafal rumus, melainkan tentang **mendefinisikan state yang tepat**. Pemilihan state yang baik menentukan kemudahan derivasi dan efisiensi memori. Selalu verifikasi rekurensi dengan tabulasi manual sebelum membuat kode.

2 Studi Kasus 1: E-Olymp 5081 – Lanterns

2.1 Deskripsi Masalah

Terdapat n lampu terpasang di sepanjang jalan. Jalan dinyatakan **aman** jika tidak ada k lampu mati yang berturut-turut. Kita diminta menghitung banyaknya konfigurasi nyala/mati lampu yang membuat jalan tetap aman, modulo $10^9 + 7$.

Batasan:

- $1 \leq n \leq 10^6$
- $1 \leq k \leq n$

2.2 Definisi State DP

State DP – Lanterns

$dp[i]$ = banyaknya konfigurasi valid untuk i lampu pertama

2.3 Penurunan Relasi Rekurensi

Ide dasarnya: lampu ke- i bisa **nyala** atau **mati**, sehingga:

$$dp[i] = 2 \times dp[i - 1]$$

Namun, perkalian ini menghitung juga konfigurasi tidak valid, yaitu ketika tepat k lampu terakhir (posisi $i - k + 1$ hingga i) semuanya mati sekaligus. Konfigurasi seperti itu harus dikurangi.

Jika tepat k lampu terakhir semua mati, maka lampu ke- $i - k$ bebas dalam keadaan apa pun — artinya ada $dp[i - k - 1]$ cara untuk sisa lampu ke-1 hingga $i - k - 1$ yang harus dikurangi. Untuk kasus $i = k$, tidak ada lampu sebelumnya sehingga hanya ada 1 konfigurasi tidak valid yang dikurangi.

Relasi Rekurensi – Lanterns

$$dp[i] = \begin{cases} 2 \times dp[i - 1] & \text{jika } i < k \\ 2 \times dp[i - 1] - 1 & \text{jika } i = k \\ 2 \times dp[i - 1] - dp[i - k - 1] & \text{jika } i > k \end{cases}$$

dengan basis $dp[0] = 1$.

Perhatian: Nilai Negatif dalam Modular Arithmetic

Operasi pengurangan dalam aritmetika modular bisa menghasilkan nilai negatif di C/C++. Selalu gunakan pola berikut setiap kali melakukan pengurangan:

$$dp[i] = (dp[i] - x + \text{MOD}) \bmod \text{MOD}$$

2.4 Verifikasi Manual ($n = 3, k = 3$)

Tabel 2: Tabulasi manual Lanterns untuk $n = 3, k = 3$

i	$2 \times dp[i - 1]$	Koreksi	$dp[i]$
0	—	—	1
1	2	—	2
2	4	—	4
3	8	-1 (kasus $i = k$)	7

Output yang diharapkan untuk $n = 3, k = 3$ adalah **7**. ✓

2.5 Kode Solusi (C)

```

1  #include <stdio.h>
2
3  #define MOD 1000000007
4
5  long long dp[1000001];
6
7  int main() {
8      int n, k;
9      scanf("%d %d", &n, &k);
10
11     dp[0] = 1;
12
13     for (int a = 1; a <= n; a++) {
14         dp[a] = (2 * dp[a - 1]) % MOD;
15
16         if (a >= k) {
17             if (a == k) {
18                 dp[a] = (dp[a] - 1 + MOD) % MOD;
19             } else {
20                 dp[a] = (dp[a] - dp[a - k - 1] + MOD) % MOD;
21             }
22         }
23     }
24
25     printf("%lld\n", dp[n]);
26     return 0;
27 }

```

Listing 1: Solusi E-Olymp 5081 Lanterns – C

2.6 Analisis Kompleksitas

Tabel 3: Kompleksitas solusi Lanterns

Aspek	Nilai
Kompleksitas Waktu	$O(n)$
Kompleksitas Ruang	$O(n)$

3 Studi Kasus 2: SPOJ AU7_5 – Students

3.1 Deskripsi Masalah

Terdapat N siswa dalam sebuah barisan, masing-masing berupa anak laki-laki (B) atau perempuan (G). Aturannya: **di antara dua B yang berurutan, harus ada minimal K anak G**. Kita diminta menghitung banyaknya barisan yang valid, modulo 5000011.

Batasan:

- $1 \leq C \leq 20$ (jumlah *test case*)
- $1 \leq N \leq 100,000$; $0 \leq K < N$

3.2 Pendekatan 1: DP 2-State

3.2.1 Definisi State

State DP – Students (2-State)

$dp[i][0]$ = jumlah barisan valid panjang i yang berakhir dengan G
 $dp[i][1]$ = jumlah barisan valid panjang i yang berakhir dengan B

3.2.2 Logika Transisi

Untuk $dp[i][0]$: posisi ke- i diisi G, sehingga posisi sebelumnya bebas berisi G atau B:

$$dp[i][0] = dp[i-1][0] + dp[i-1][1]$$

Untuk $dp[i][1]$: posisi ke- i diisi B, sehingga K posisi sebelumnya *harus* semua G. Ini berarti posisi ke- $(i - K - 1)$ adalah posisi pertama yang “bebas” kembali, sehingga kontribusinya adalah total konfigurasi valid pada panjang $i - K - 1$.

Transisi DP 2-State

$$dp[i][1] = \begin{cases} dp[i-1][0] + dp[i-1][1] & \text{jika } i - K - 1 > 0 \\ 1 & \text{jika } i - K - 1 \leq 0 \end{cases}$$

Catatan. Kondisi $i - K - 1 \leq 0$ berarti belum ada cukup ruang untuk menempatkan dua B sejauh K posisi, sehingga hanya ada 1 cara valid menempatkan B di posisi ke- i (yaitu dengan semua posisi sebelumnya berisi G).

3.2.3 Tabulasi DP 2-State ($N = 4$, $K = 2$)

Tabel 4: Tabulasi DP 2-state untuk $N = 4$, $K = 2$

i	$dp[i][0]$ (akhiran G)	$dp[i][1]$ (akhiran B)	Total	Contoh Barisan Valid
1	1	1	2	G, B
2	2	1	3	GG, BG, GB
3	3	1	4	GGG, BGG, GBG, GGB
4	4	2	6	GGGG, BGGG, GBGG, GGBG, GGGB, BGGB

3.2.4 Kode DP 2-State (C++)

```

1  #include <algorithm>
2  #include <cstdio>
3  using namespace std;
4  const int MOD = 5000011, MAXN = 100001;
5  int dp[MAXN][2];
6
7  int main() {
8      int t, n, k;
9      scanf("%d", &t);
10     while (t--) {
11         scanf("%d %d", &n, &k);
12         dp[1][0] = dp[1][1] = 1;
13         for (int i = 2; i <= n; ++i) {
14             dp[i][0] = (dp[i-1][0] + dp[i-1][1]) % MOD;
15             if (i - k - 1 > 0) {
16                 dp[i][1] = (dp[i-k-1][0] + dp[i-k-1][1]) % MOD;
17             } else {
18                 dp[i][1] = 1;
19             }
20         }
21         printf("%d\n", (dp[n][0] + dp[n][1]) % MOD);
22     }
23     return 0;
24 }

```

Listing 2: Solusi SPOJ Students – DP 2-State (C++)

3.3 Pendekatan 2: DP 1-State (Optimasi)

3.3.1 Motivasi Penyederhanaan

Dari tabel tabulasi, perhatikan bahwa yang kita butuhkan di akhir adalah:

$$\text{Total}[i] = dp[i][0] + dp[i][1]$$

Substitusikan persamaan transisi ke dalam $\text{Total}[i]$:

$$\begin{aligned}
 \text{Total}[i] &= dp[i][0] + dp[i][1] \\
 &= (dp[i-1][0] + dp[i-1][1]) + (dp[i-k-1][0] + dp[i-k-1][1]) \\
 &= \text{Total}[i-1] + \text{Total}[i-k-1]
 \end{aligned}$$

Reduksi ke DP 1-State

$$\text{Total}[i] = \text{Total}[i-1] + \text{Total}[i-k-1]$$

Dalam kode, cukup gunakan array satu dimensi $dp[i]$ yang merepresentasikan $\text{Total}[i]$.

3.3.2 Bukti Nilai Awal ($dp[i] = i + 1$ saat $i - k - 1 \leq 0$)

Saat $i - k - 1 \leq 0$, belum mungkin ada dua B yang berjarak K posisi. Artinya semua barisan dengan **paling banyak satu** B adalah valid. Banyaknya barisan panjang i dengan tepat satu B adalah i (karena B bisa berada di posisi mana saja), ditambah satu barisan semua G. Maka:

$$dp[i] = i + 1$$

Verifikasi dengan tabel: $dp[1] = 2$, $dp[2] = 3$, $dp[3] = 4$ (untuk $K = 2$). ✓

3.3.3 Perbandingan Jumlah Entri

Tabel 5: Perbandingan entri array DP 2-state vs DP 1-state

Pendekatan	Jumlah Entri	Keterangan
DP 2-State	$2N$	Array $N \times 2$
DP 1-State	N	Array satu dimensi

3.3.4 Tabulasi DP 1-State ($N = 4$, $K = 2$)

Tabel 6: Tabulasi DP 1-state (Total[i]) untuk $N = 4$, $K = 2$

i	Total[$i - 1$]	Total[$i - k - 1$]	$dp[i]$
1	—	—	2 ($= 1 + 1$)
2	2	—	3 ($= 2 + 1$)
3	3	—	4 ($= 3 + 1$)
4	4	Total[1] = 2	6 ($= 4 + 2$)

3.3.5 Kode DP 1-State (C)

```

1 #include <stdio>
2 #define mod 5000011
3 int dp[100001];
4
5 int main() {
6     int c, n, k;
7     scanf("%d", &c);
8     while (c--) {
9         scanf("%d %d", &n, &k);
10        dp[1] = 2;
11        for (int i = 2; i <= n; i++) {
12            if (i - k - 1 > 0)
13                dp[i] = (dp[i-1] + dp[i-k-1]) % mod;
14            else
15                dp[i] = i + 1;
16        }
17        printf("%d\n", dp[n] % mod);
18    }

```



```

19     return 0;
20 }

```

Listing 3: Solusi SPOJ Students – DP 1-State (C)

3.4 Perbandingan Dua Pendekatan

Tabel 7: Perbandingan DP 2-State vs DP 1-State untuk masalah Students

Aspek	DP 2-State	DP 1-State
Dimensi array	$N \times 2$	N
Kompleksitas waktu	$O(N)$	$O(N)$
Kompleksitas ruang	$O(2N)$	$O(N)$
Kemudahan membaca	Eksplisit, mudah dipahami	Ringkas
Cara derivasi	Langsung dari state	Perlu substitusi aljabar

3.5 Kaitan Antar Studi Kasus

Tabel 8: Perbandingan dua studi kasus pada Pertemuan 7

Aspek	Lanterns	Students
Masalah	Lampu mati berurutan	Barisan siswa B dan G
State	$dp[i]$ = konfigurasi valid ke- i	$dp[i]$ = barisan valid panjang i
Rekurensi	$2 \cdot dp[i - 1] - dp[i - k - 1]$	$dp[i - 1] + dp[i - k - 1]$
Operasi kunci	Pengurangan (kurangi invalid)	Penjumlahan (gabungkan valid)
Modulo	$10^9 + 7$	5,000,011
Bahasa kode	C	C / C++

4 Sintesis: Pola Berpikir Pertemuan 7

4.1 Hierarki Teknik yang Dipelajari

Berikut hierarki teknik yang berkembang selama pertemuan ini, dari paling dasar ke paling spesifik:

1. **Identifikasi sifat DP** — pastikan masalah memiliki *overlapping subproblems* dan *optimal substructure*.
2. **Definisi State** — tentukan apa yang diwakili oleh $dp[i]$ secara presisi.
3. **Penurunan Rekurensi** — rumuskan transisi dari state ke state berikutnya berdasarkan pilihan yang tersedia.
4. **Penentuan Basis** — identifikasi nilai-nilai awal $dp[0]$, $dp[1]$, dan kondisi tepi lainnya.
5. **Verifikasi Manual** — selalu cek rekurensi dengan tabulasi kasus kecil sebelum membuat kode.

6. **Penanganan Modular** — gunakan pola $(a - b + \text{MOD}) \bmod \text{MOD}$ untuk pengurangan.
7. **Reduksi State** — jika memungkinkan, sederhanakan DP multi-state ke satu state melalui substitusi aljabar untuk menghemat memori.

5 Checklist Konsep yang Harus dikuasai

1. **Kapan DP digunakan:** *overlapping subproblems* dan *optimal substructure*.
2. **Relasi Lanterns:** $dp[i] = 2 \cdot dp[i - 1] - dp[i - k - 1]$, dengan tiga kasus ($i < k$, $i = k$, $i > k$).
3. **Penanganan nilai negatif:** selalu tambahkan MOD sebelum operasi modulo: $(a - b + \text{MOD}) \bmod \text{MOD}$.
4. **DP 2-State Students:** pisahkan state berdasarkan karakter terakhir (G atau B).
5. **Reduksi DP 1-State:** $\text{Total}[i] = \text{Total}[i - 1] + \text{Total}[i - k - 1]$ diperoleh via substitusi aljabar.
6. **Nilai awal DP 1-State:** $dp[i] = i + 1$ saat $i - k - 1 \leq 0$ karena belum mungkin ada dua B sejauh K posisi.
7. **Verifikasi:** selalu cek formula dengan tabulasi manual kasus kecil sebelum implementasi.

6 Kesimpulan

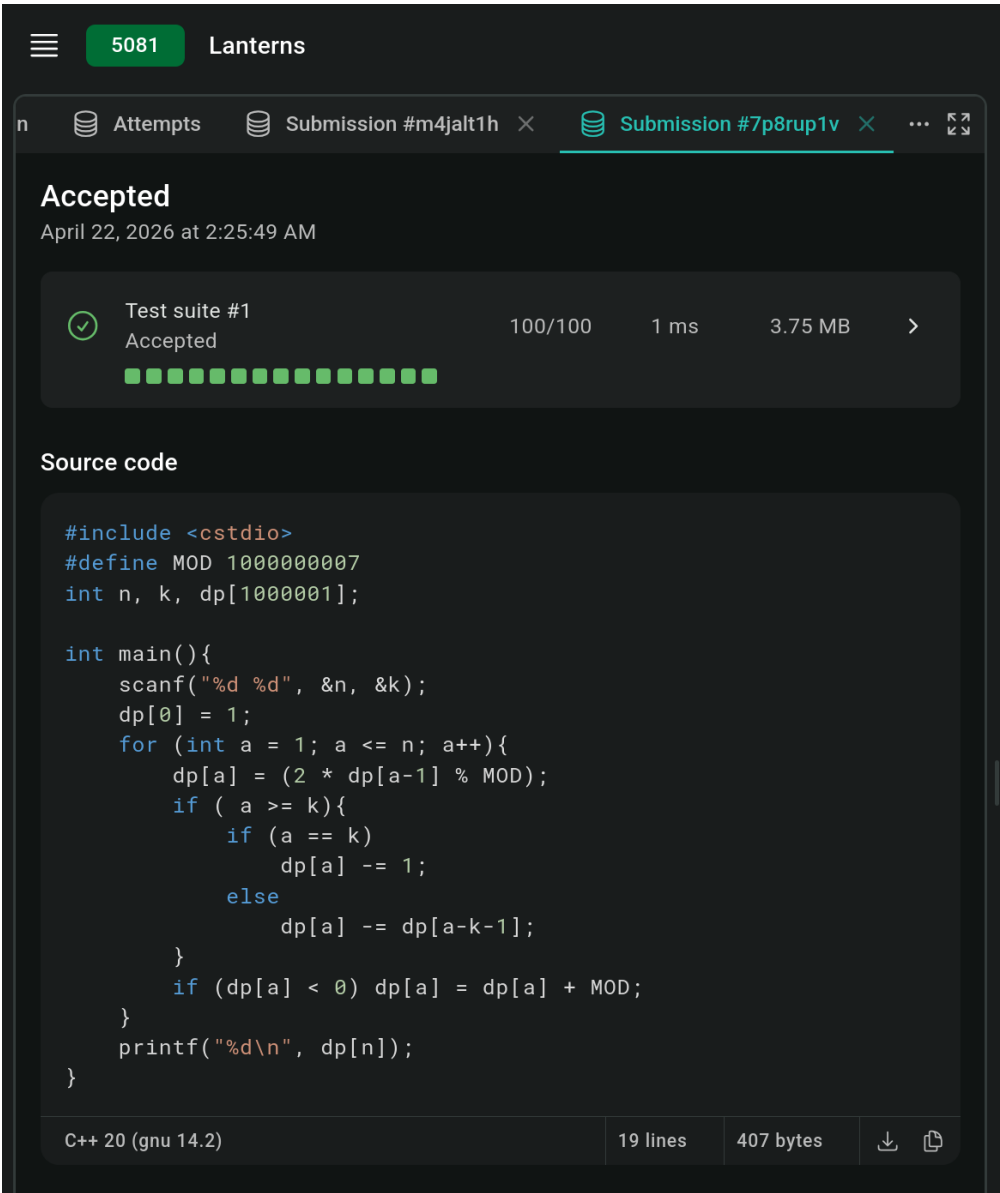
1. **DP sebagai alat utama untuk submasalah tumpang tindih.** Ketika permasalahan tidak memiliki *closed form* yang jelas dan submasalah dihitung berulang, DP menjadi pilihan pertama dengan kompleksitas waktu yang terkendali.
2. **Identifikasi state adalah kunci.** Memilih state yang tepat menentukan kemudahan derivasi dan efisiensi memori. DP 2-state lebih mudah dipahami; DP 1-state lebih efisien setelah substitusi aljabar berhasil dilakukan.
3. **Reduksi state membuka efisiensi lebih lanjut.** Analisis pola tabulasi — bukan sekadar menghafal rumus — sering kali membuka peluang penyederhanaan. Rumus $\text{Total}[i] = \text{Total}[i - 1] + \text{Total}[i - k - 1]$ pada Students diperoleh murni dari substitusi aljabar definisi state.
4. **Penanganan modular harus konsisten.** Pengurangan dalam *modular arithmetic* berpotensi menghasilkan bilangan negatif. Pola $(a - b + \text{MOD}) \bmod \text{MOD}$ harus diterapkan secara konsisten di seluruh kode.

7 References

Pustaka

- [1] Rosen, K. H. (2012). *Discrete Mathematics and Its Applications* (7th ed.). McGraw-Hill. Bab 8: Linear Recurrence Relations.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [3] E-Olymp. (n.d.). *Problem 5081 – Lanterns*. <https://www.e-olymp.com/en/problems/5081>
- [4] SPOJ. (n.d.). *AU7_5 – Students*. https://www.spoj.com/problems/AU7_5/
- [5] Sulaiman, R. (2026). *Materi Kuliah PAA Pertemuan Ke-7*. Institut Teknologi Sepuluh Nopember.

8 Bukti Submission



Gambar 1: Accepted submission – E-Olymp 5081 Lanterns

GilbranNRP134: submissions
STUDENTS

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35679224	2026-04-15 17:56:34	STUDENTS	accepted 0.01 / 5.2M	0.01	5.2M	CPP14

Gambar 2: Accepted submission – SPOJ AU7_5 Students